
74
W hich Animal
31
Ga ve Us SARS?
Evolutionary tree reconstruction
Global Outbreak
Stopped in its
Tracks
Bioinformatics
Saves the Day

Trouble at the Metropole Hotel

On February 21, 2003, a Chinese doctor named Liu Jianlun flew to Hong Kong to attend a wedding. He checked into Room 911 of the Metropole Hotel, and although he felt unwell, he spent the day sightseeing. The next day, he became too ill to attend the wedding and was admitted to a hospital. Two weeks later, Dr. Jianlun was dead.

On his deathbed, Jianlun told doctors that he had recently treated sick patients in Guangdong Province, China where a deadly, highly contagious respiratory illness had infected hundreds of people. The Chinese government had made brief mention of this incident to the World Health Organization but had concluded that the likely culprit was a common bacterial infection. By the time anyone realized the severity of the disease, it was already too late to stop the outbreak. On February 23, a man who had stayed across the hall from Dr. Jianlun at the Metropole traveled to Hanoi and died after infecting 80 people. On February 26, a woman checked out of the Metropole, traveled back to Toronto, and died after initiating an outbreak there. On March 1, a third guest was admitted to a hospital in Singapore, where sixteen additional cases of the illness arose within two weeks.

Consider that it took four years for the Black Death, which killed over a third of all Europeans in the 14th Century, to travel from Constantinople to Kiev. Or that HIV took two decades to circle the globe. In contrast, this mysterious new disease had crossed the Pacific Ocean within a week of entering Hong Kong.

As health officials braced for the impact of the fastest-traveling pandemic in human history, panic set in. Businesses were closed, sick passengers were removed from airplanes, and Chinese officials threatened to execute infected patients who violated quarantine.

International travel may have helped the disease spread rapidly, but international collaboration would eventually contain it. In a matter of a few weeks, biologists identified that a virus had caused the epidemic and sequenced its genome. In the process, the mysterious new disease earned a name: **Severe Acute Respiratory Syndrome**, or **SARS**.

The evolution of SARS

The virus causing SARS belongs to a family of viruses called **coronaviruses**, which are named after the Latin *corona* (meaning “crown”) because the virus particle resembles the sun’s corona (see the figure below). Coronaviruses infect the respiratory tracts of mammals and birds and typically cause only minor problems, like the common cold. Before SARS, no one believed that a coronavirus could wreak such havoc.

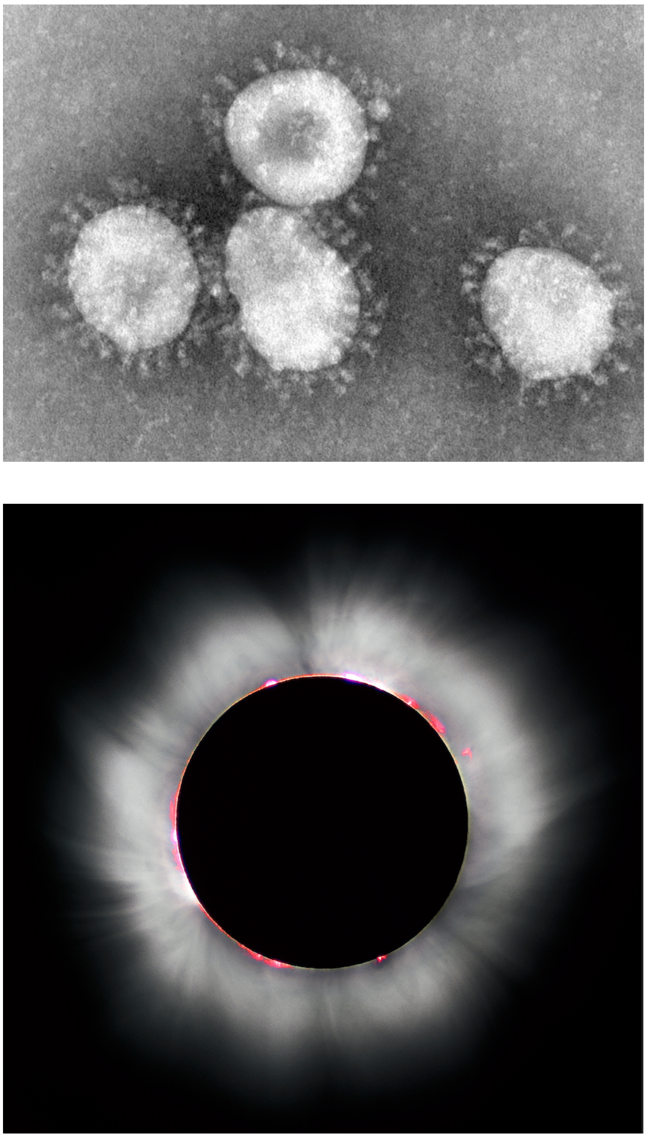


Figure: (Top) Coronavirus particles. (Bottom) A solar eclipse with the sun’s corona visible. Courtesy: Luc Viatour.

The Fastest Outbreak | Step 4

Coronaviruses, influenza viruses, and HIV are all **RNA viruses**, meaning that they possess RNA instead of DNA. RNA replication has a higher error rate than DNA replication, and so RNA viruses are capable of mutating more quickly into divergent strains. The rapid mutation of RNA viruses explains why the flu shot changes from year to year and why there are many different subtypes of HIV.

SARS researchers initially hypothesized that, like HIV and influenza, the **SARS coronavirus** (abbreviated as **SARS-CoV**) had jumped from animals to humans. They first named birds as the likely suspect because of the similarities between the SARS outbreak and “bird flu”, a form of influenza originating in chickens that is difficult to transmit to humans but is even deadlier than SARS, killing over half the people it infects. Yet when researchers sequenced the 29,751 nucleotide-long SARS-CoV genome in April 2003, it became evident that SARS did not come from birds because its genome did not resemble avian coronaviruses.

By fall 2003, researchers had sequenced many SARS-CoV strains from patients in various countries, but many questions still remained unanswered. How did SARS-CoV cross the species barrier to humans? When and where did it happen? How did SARS spread around the world, and who infected whom?

The Fastest Outbreak | Step 5

Each of these questions about SARS is ultimately related to the problem of constructing **evolutionary trees** (also known as **phylogenies**). For another example, by constructing an evolutionary tree of primate viruses related to HIV (figure below), we can infer that HIV was transmitted to humans on five separate occasions (see DETOUR: When Did HIV Jump From Primates to Humans? (https://stepic.org/lesson/Detour-When-Did-HIV-Jump-from-Primates-to-Humans-10338)). But what algorithm did we use to construct this phylogeny?

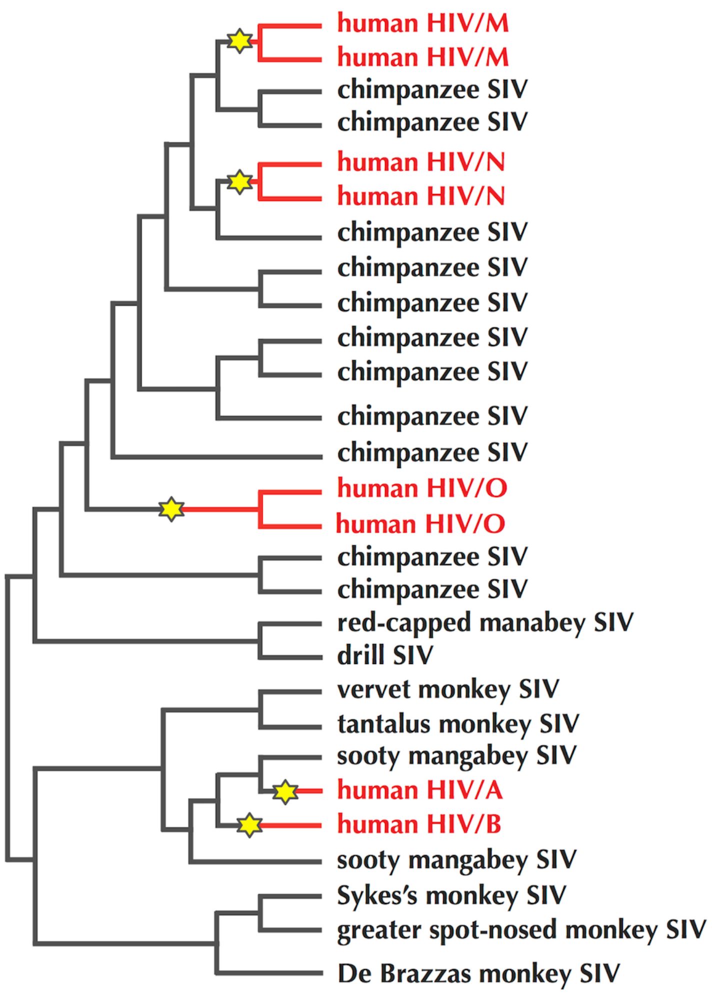


Figure: HIV comprises five different viral families, denoted as A, B, M, N, and O, with the M family responsible for 95% of all HIV infections. The five families are different offshoots of the evolutionary tree for the Simian Immunodeficiency Virus (SIV), which infects primates. Stars indicate transitions from primates to humans. The A and B families originated in sooty mangabey monkeys, whereas the M, N, and O families originated in chimpanzees.

Constructing a distance matrix from multiple coronavirus genomes

To determine how SARS jumped from animals to humans, scientists started sequencing coronaviruses from various species in order to determine which one is the most similar to SARS-CoV. However, constructing a multiple alignment of entire viral genomes is tricky because viral genes are often rearranged, inserted, and deleted. For this reason, scientists focused on only one of the six genes in SARS-CoV. This gene encodes the **Spike protein**, which identifies and binds to receptor sites on the host's cell membrane.

In SARS-CoV, the Spike protein is 1,255 amino acids long and has rather weak similarity with Spike proteins in other coronaviruses. However, even these subtle similarities turned out to be sufficient for constructing a multiple alignment of Spike proteins across various coronaviruses.

Transforming Distance Matrices into Evolutionary Trees | Step 2

After constructing a multiple alignment of genes from n different species, biologists often transform this alignment into an n x n **distance matrix** D. In many cases, $D_{i,j}$ represents the number of differing symbols between the genes representing the i-th and j-th rows of the alignment (see the figure below). However, distance matrices can be constructed using a variety of different distance functions in order to suit different applications. For example, $D_{i,j}$ could also represent the edit distance between genes from the i-th and j-th species. Or, a distance matrix for n genomes could be constructed from the 2-break distances between each pair of genomes.

Species	Alignment	Distance Matrix			
		Chimp	**Human**	**Seal**	**Whale**
Chimp	ACGTAGGCCT	0	3	6	4
Human	ATGTAAGACT	3	0	7	5
Seal	TCGAGAGCAC	6	7	0	2
Whale	TCGAAAGCAT	4	5	2	0

Figure: A multiple alignment of hypothetical DNA sequences from four species, along with the distance matrix produced by counting the number of differing symbols between each pair of rows in this multiple alignment.

Transforming Distance Matrices into Evolutionary Trees | Step 3

Regardless of which distance function we use, in order to be a distance matrix, *D* must satisfy three properties. It must be **symmetric** (for all *i* and *j*, $D_{i,j} = D_{j,i}$), **non-negative** (for all *i* and *j*, $D_{i,j} \geq 0$) and satisfy the **triangle inequality** (for all *i*, *j*, and *k*, $D_{i,j} + D_{j,k} \geq D_{i,k}$).

Exercise Break: Prove that if $D_{i,j}$ is equal to the number of differing symbols between the *i*-th and *j*-th rows in a multiple alignment, then *D* is symmetric, non-negative, and satisfies the triangle inequality.

By the end of 2003, bioinformaticians had sequenced many coronaviruses taken from a variety of animals and SARS patients and then computed the associated distance matrix. They needed to use this information in order to construct a coronavirus phylogeny and understand the origin and spread of the SARS epidemic.

Evolutionary trees as graphs

You may have noticed that the HIV tree in the Figure on step 2 has the structure of a graph. Furthermore, the figure below shows the representation of the phylogeny of all life as a graph.

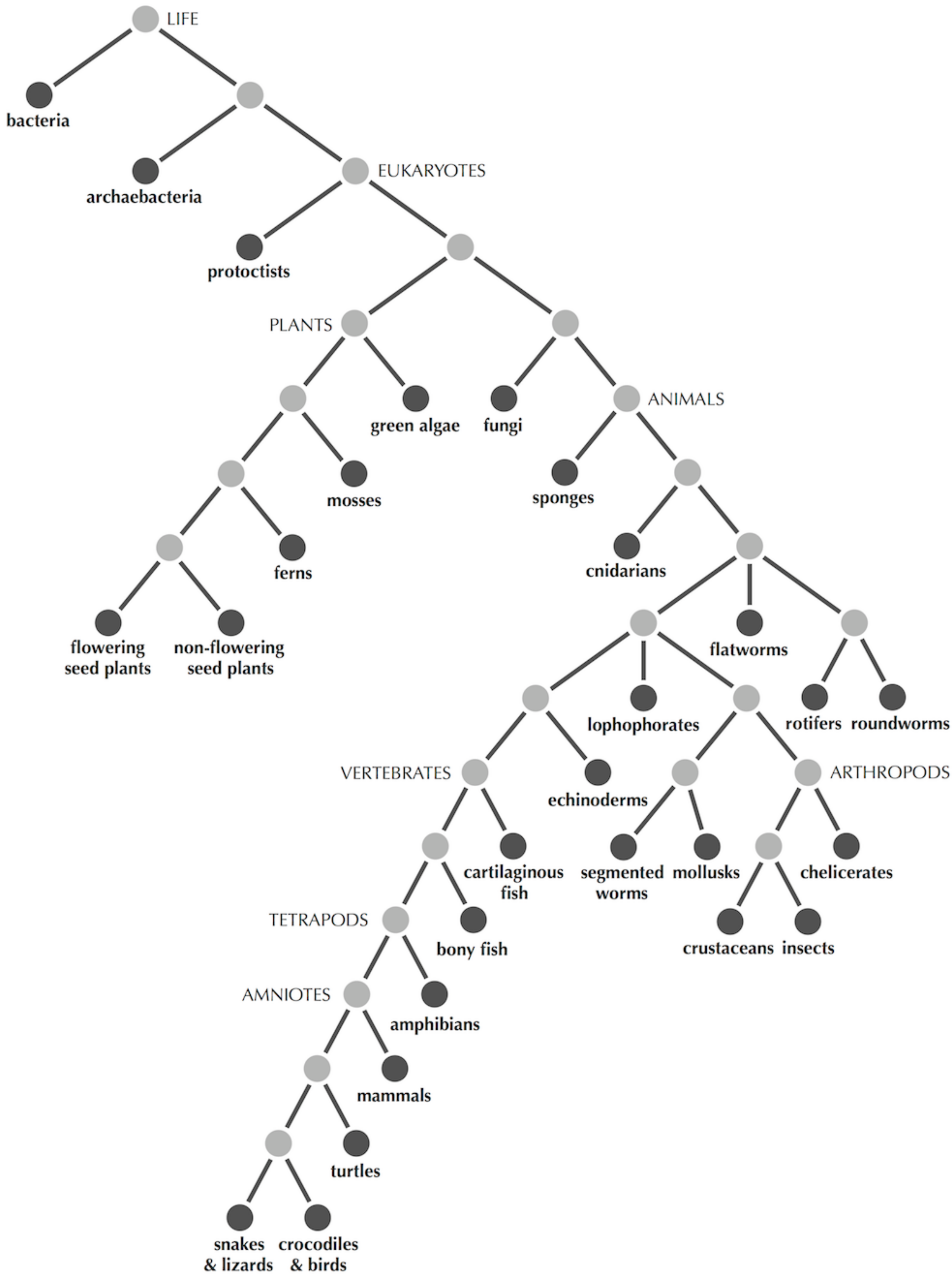


Figure: A connected graph without cycles that models an evolutionary tree of all life on Earth. Present-day species are shown as darker nodes.

Transforming Distance Matrices into Evolutionary Trees | Step 5

Graphs used to model phylogenies share two properties. They are connected (i.e. it is possible to reach any node from any other node), and they contain no cycles. For this reason, we will define a **tree** as a connected graph without cycles.

See the figure below for a few additional examples; the darker nodes are **leaves**, or nodes having degree 1 (in DETOUR: Graphs (https://stepic.org/lesson/Detour-An-Introduction-to-Graphs-208/), we defined the degree of a node as the number of edges connected to that node). Nodes of larger degree are called **internal nodes**.

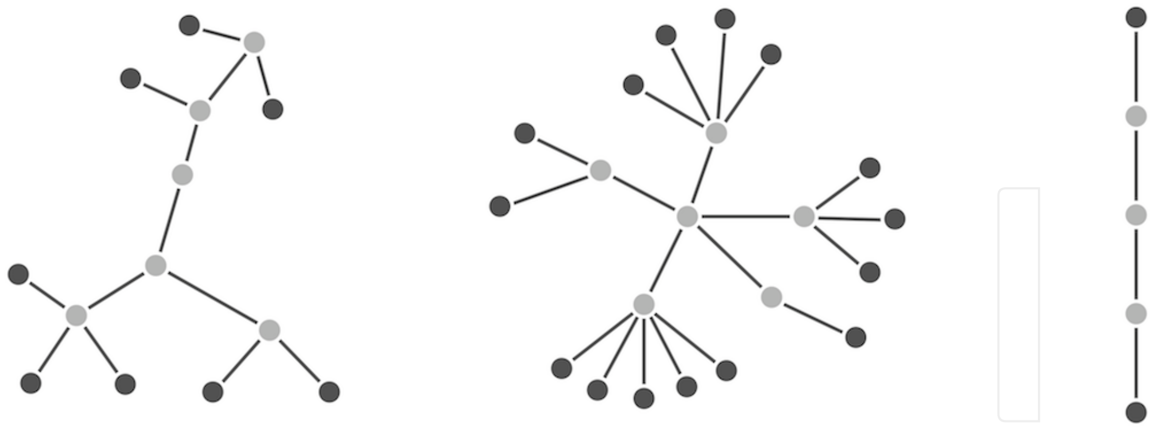


Figure: Trees come in a variety of different shapes. In each of the three trees shown, leaves have been drawn darker than internal nodes.

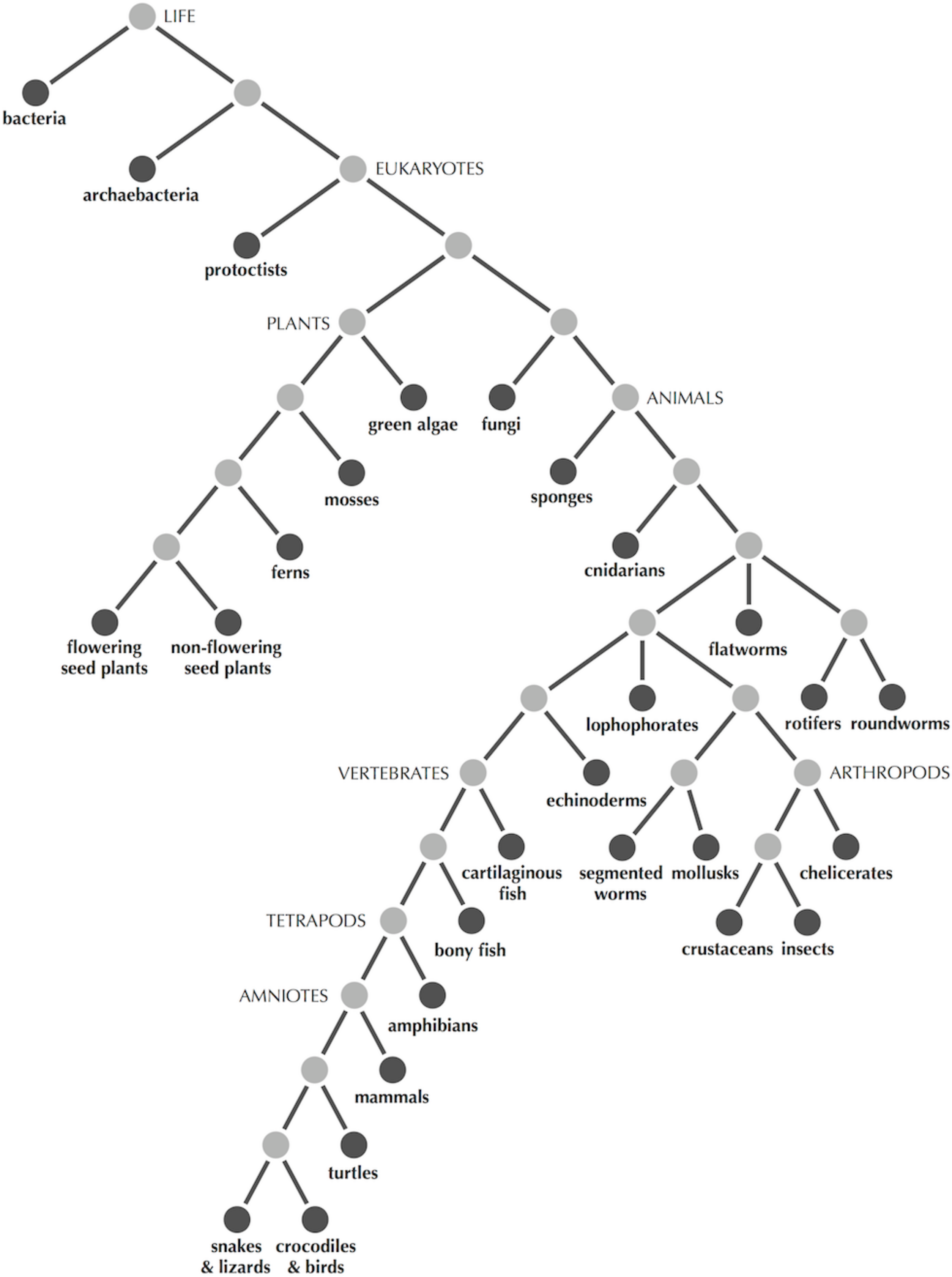
Exercise Break: Prove the following statements:

- Every tree with at least two nodes has at least two leaves.
- Every tree with *n* nodes has *n* - 1 edges;
- there exists exactly one path connecting every pair of nodes in a tree. Hint: what would happen if there were two different paths connecting a pair of nodes? What would happen if there were no paths connecting a pair of nodes?

Transforming Distance Matrices into Evolutionary Trees | Step 6

Take another look at the tree of life figure, reproduced below. You will see that present-day species have been assigned to the leaves of this tree. Internal nodes represent unknown ancestor species.

Given a leaf *j*, there is only one node connected to *j* by an edge, which we call the **parent** of *j*, denoted *Parent*(*j*). An edge connecting a leaf to its parent is called a **limb**.


LIFE
bacteria
archaebacteria
EUKARYOTES
protoctists
PLANTS
green algae
fungi
ANIMALS
mosses
sponges
ferns
cnidarians
flowering seed plants
non-flowering seed plants
flatworms
lophophorates
rotifers
roundworms
VERTEBRATES
echinoderms
ARTHROPODS
cartilaginous fish
segmented worms
mollusks
chelicerates
TETRAPODS
bony fish
crustaceans
insects
AMNIOTES
amphibians
mammals
turtles
snakes & lizards
crocodiles & birds

Transforming Distance Matrices into Evolutionary Trees | Step 7

In a **rooted tree**, one node is designated as a special node called the **root**, and the edges in the tree automatically inherit an implicit orientation away from the root (see the figure below). This edge orientation models time: the ancestor of all species in the tree is found at the root, and evolution proceeds from the root outward through the tree. Trees without a designated root are called **unrooted**.

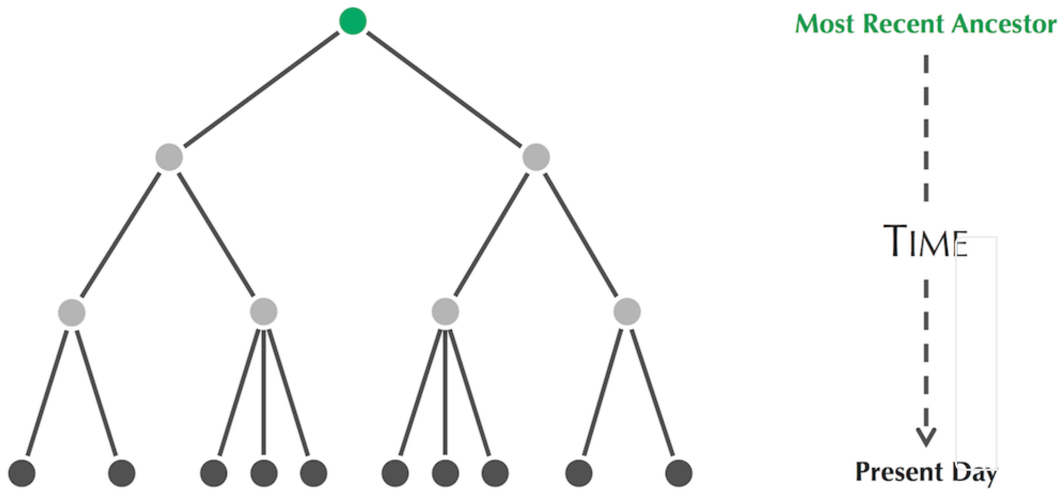
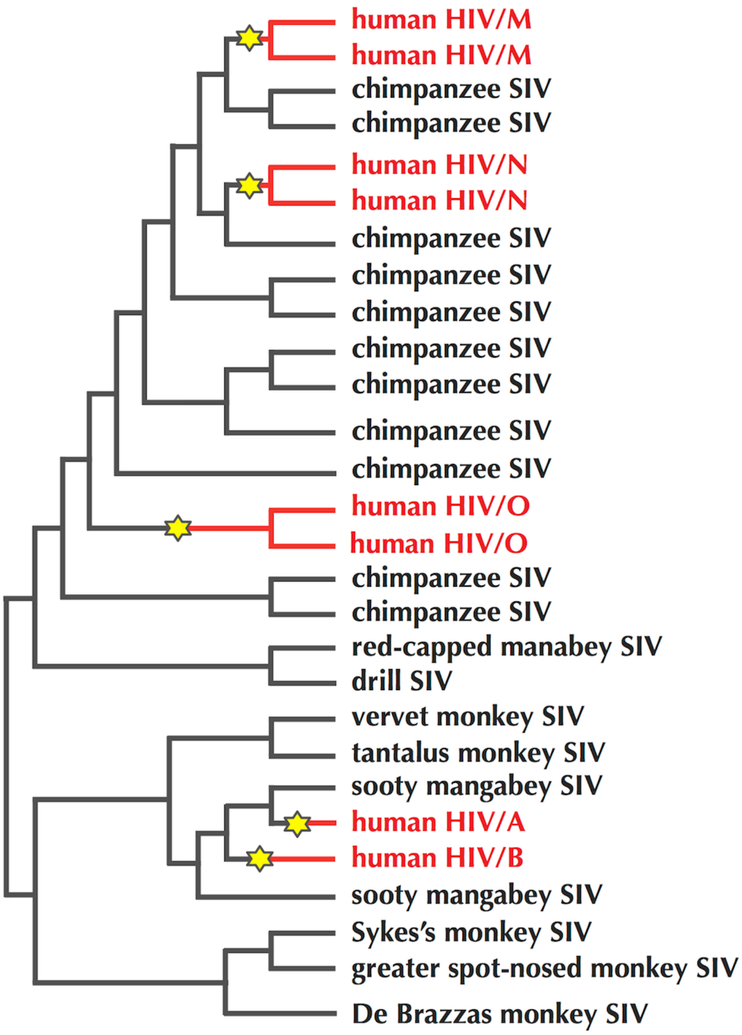


Figure: A rooted tree, with the root (representing an ancestor of all species in the tree) indicated in green at the top of the tree. The presence of the root implies an orientation of edges in the tree away from the root.

Transforming Distance Matrices into Evolutionary Trees | Step 8

STOP and Think: The HIV phylogeny we encountered before is reproduced below. Where would you place the root in this phylogeny?



Transforming Distance Matrices into Evolutionary Trees | Step 9

In this chapter, we will analyze rooted trees when we will attempt to infer the node corresponding to the ancestor of all species in the tree; otherwise, we will analyze unrooted trees. The figure below shows an unrooted tree of HIV viruses produced from a different dataset than the one used to create the figure in the previous step. By proposing two additional subtypes of HIV, it illustrates that the classification of HIV into five families is not written in stone.

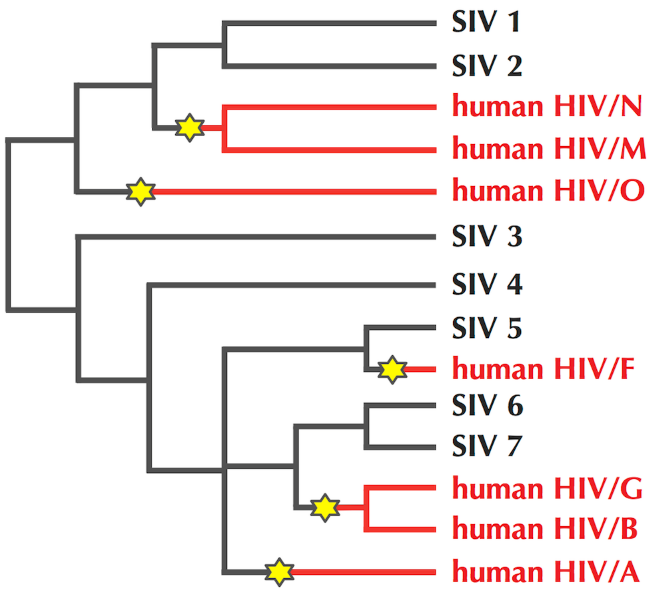


Figure: An unrooted tree of HIV and SIV viruses that suggests additional viral families F and G in addition to the viral families A, B, M, N, and O shown in the figure on the previous step.

Distance-based phylogeny construction

We will first focus on deriving an unrooted tree from a distance matrix. The leaves of this tree should correspond to the species represented by the matrix (with internal nodes corresponding to unknown ancestral species). To reflect the evolutionary distance between species in a tree, we assign each edge a non-negative weight representing the distance between the organisms that the edge connects, as shown in the figure below.

SPECIES	ALIGNMENT	DISTANCE MATRIX			
		Chimp	**Human**	**Seal**	**Whale**
Chimp	ACGTAGGCCT	0	3	6	4
Human	ATGTAAGACT	3	0	7	5
Seal	TCGAGAGCAC	6	7	0	2
Whale	TCGAAAGCAT	4	5	2	0

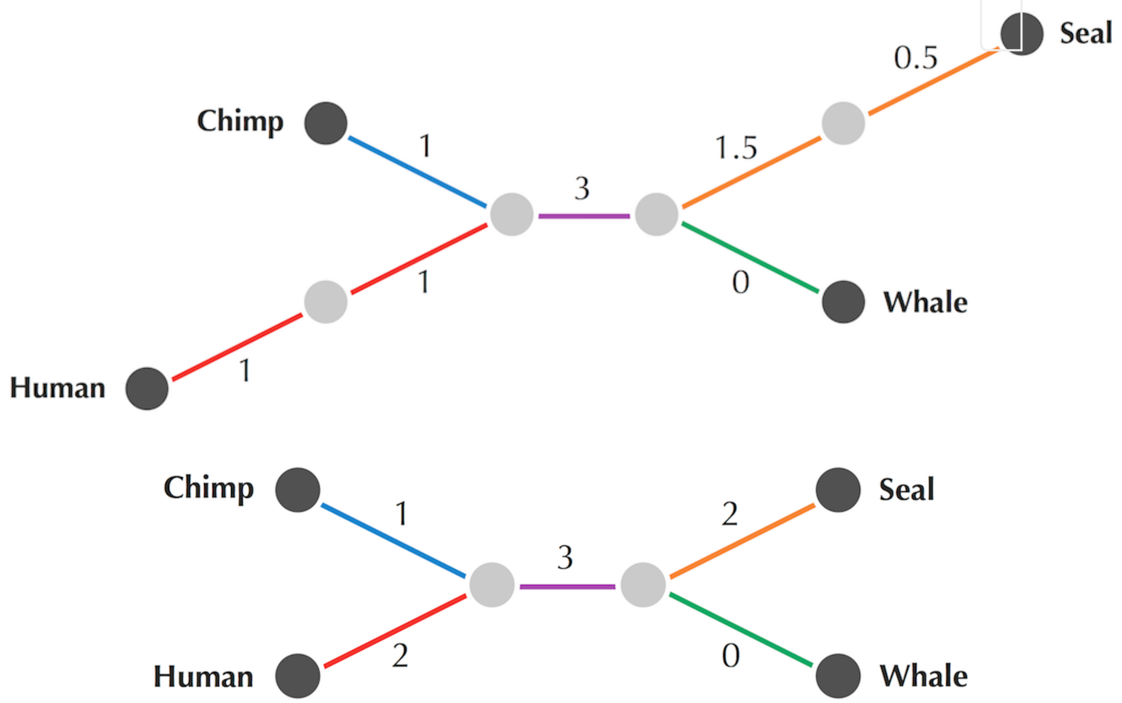


Figure: The toy distance matrix constructed from a multiple alignment as well as two unrooted trees fitting this distance matrix.

Transforming Distance Matrices into Evolutionary Trees | Step 11

In this chapter, we define the length of a path in a tree as the sum of the lengths of its edges (rather than the number of edges on the path). As a result, the evolutionary distance between two present-day species corresponding to leaves *i* and *j* in a tree *T* is equal to the length of the unique path connecting *i* and *j*, denoted $d_{i,j}(T)$.

Distances Between Leaves Problem: *Compute the distances between leaves in a weighted tree.*

Input: An integer *n* followed by the adjacency list of a weighted tree with *n* leaves.
Output: An *n* x *n* matrix ($d_{i,j}$), where $d_{i,j}$ is the length of the path between leaves *i* and *j*.

CODE CHALLENGE: Solve the Distances Between Leaves Problem. The tree is given as an adjacency list of a graph whose leaves are integers between 0 and *n* - 1; the notation *a*->*b*:*c* means that node *a* is connected to node *b* by an edge of weight *c*. The matrix you return should be space-separated.

Extra Dataset [(http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Distance_Between_Leaves.txt)](http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Distance_Between_Leaves.txt)

Sample Input:

```
4
0->4:11
1->4:2
2->5:6
3->5:7
4->0:11
4->1:2
4->5:4
5->4:4
5->3:7
5->2:6
```

Sample Output:

```
0	13	21	22
13	0	12	13
21	12	0	13
22	13	13	0
```

Time Limit: `5 mins`

To pass this quiz please visit stepic.org/lesson/-10328/step/11

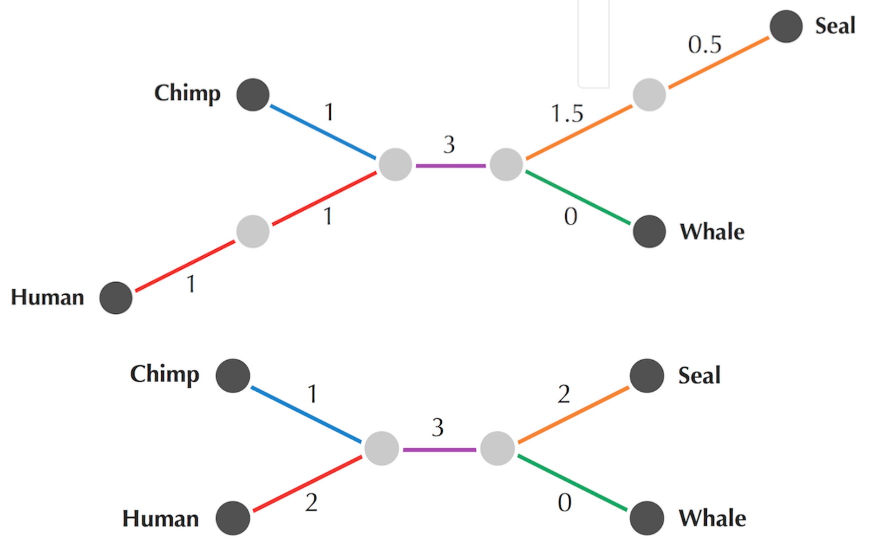
Transforming Distance Matrices into Evolutionary Trees | Step 12

The Distance Between Leaves Problem is straightforward to solve, but we would like to solve the reverse problem, in which we must produce an unrooted tree that models a given distance matrix. We say that a weighted unrooted tree *T* **fits** a distance matrix *D* if $d_{i,j}(T) = D_{i,j}$ for every pair of leaves *i* and *j* (the figure below shows two trees fitting the toy distance matrix that we encountered before).

Distance-Based Phylogeny Problem: *Reconstruct an evolutionary tree fitting a distance matrix.*
  Input: A distance matrix.
  Output: A tree fitting this distance matrix.

STOP and Think: Does the Distance-Based Phylogeny Problem always have a solution?

SPECIES	ALIGNMENT	DISTANCE MATRIX			
		Chimp	**Human**	**Seal**	**Whale**
Chimp	ACGTAGGCCT	0	3	6	4
Human	ATGTAAGACT	3	0	7	5
Seal	TCGAGAGCAC	6	7	0	2
Whale	TCGAAAGCAT	4	5	2	0



Transforming Distance Matrices into Evolutionary Trees | Step 13

Not every distance matrix has a tree fitting it (see DETOUR: Searching for a Tree Fitting a Distance Matrix (http://stepic.org/lesson/Detour-Searching-for-a-Tree-Fitting-a-Distance-Matrix-10339/)). We therefore call a distance matrix **additive** if there exists a tree that fits this matrix and **non-additive** otherwise (an example 4 x 4 non-additive matrix is shown below).

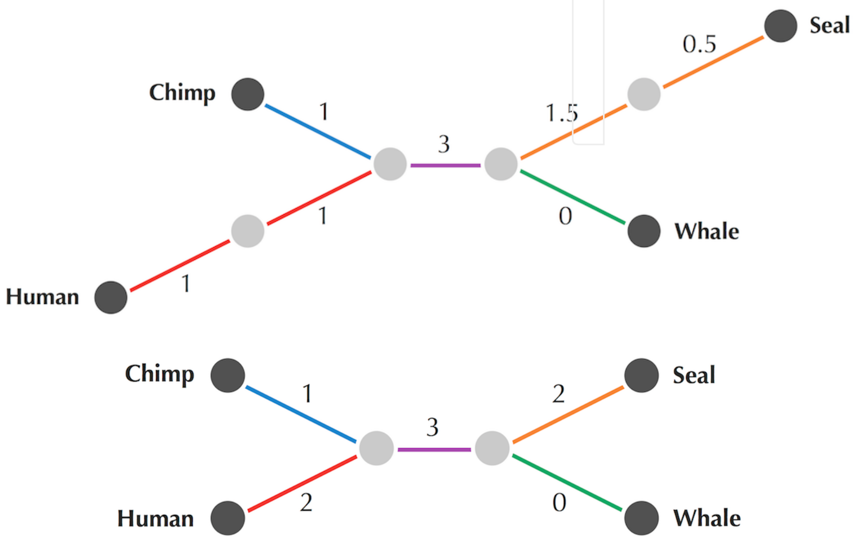
The term “additive” is used because the sum of lengths of all edges along the path between leaves *i* and *j* in the additive tree adds to $D_{i,j}$.

	i	***j***	***k***	***l***
i	0	3	4	3
j	3	0	4	5
k	4	4	0	2
l	3	5	2	0

Transforming Distance Matrices into Evolutionary Trees | Step 14

Both trees below fit the toy distance matrix, so it would be nice to have a notion of a “canonical” tree fitting a distance matrix. Extending definitions introduced in a previous chapter to undirected graphs, we say that a path in a tree is **non-branching** if every node other than the beginning and ending node of the path has degree equal to 2. A non-branching path is **maximal** if it is not a subpath of an even longer non-branching path. If we substitute every maximal non-branching path by a single edge whose length is equal to the length of the path, then the tree in figure on top becomes the tree in figure on the bottom. In general, after such a transformation, there are no nodes of degree 2; a tree satisfying this property is called a **simple tree**.

Species	Alignment	Distance Matrix			
		Chimp	**Human**	**Seal**	**Whale**
Chimp	ACGTAGGCCT	0	3	6	4
Human	ATGTAAGACT	3	0	7	5
Seal	TCGAGAGCAC	6	7	0	2
Whale	TCGAAAGCAT	4	5	2	0



Transforming Distance Matrices into Evolutionary Trees | Step 15

It turns out that if a matrix is additive, then there exists a *unique* simple tree fitting this matrix. In the Distance-Based Phylogeny Problem, we will therefore use the terminology *Tree*(*D*) to denote the simple tree fitting the additive distance matrix *D*. Our question, then, is how to construct *Tree*(*D*) from *D*.

Exercise Break: Prove that every simple tree with *n* leaves has at most *n* - 2 internal nodes.

A quest for neighboring leaves

A natural first step for solving the Distance-Based Phylogeny Problem would be to ensure that the two closest species with respect to the distance matrix D correspond to neighbors in *Tree*(D). In other words, that the minimum value $D_{i,j}$ corresponds to leaves i and j having the same parent. In the rest of this chapter, when we refer to the minimum element of a matrix, we are referring to a minimum **off-diagonal** element, i.e., a value $D_{i,j}$ such that $i \neq j$.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 2

Theorem: Every simple tree with at least three nodes has a pair of neighboring leaves.

Proof: Given a simple tree T with at least three nodes, consider a path $P = (v_1, \ldots, v_k)$ that has the maximum number of nodes of any path in T. Because T has at least three nodes, k must be at least 3. Furthermore, nodes v_1 and v_k must be leaves, since otherwise we could extend P into a longer path. Because T is simple, node v_2, which is the parent of v_1, must have at least three adjacent nodes: v_1, v_3, and yet another node w.

We claim that w is a leaf, which would imply that leaves v_1 and w are neighbors. We will proceed by contradiction: if w were not a leaf, then since T is simple, w would have degree at least 3 and therefore be adjacent to another node u. As a result, we could form the path $P' = (u, w, v_2, v_3, \ldots, v_k)$, which contains $k + 1$ nodes and contradicts our original assumption that P has the maximum number of nodes. Thus, w must be a leaf, implying that v_1 and w are neighbors and establishing the result. □

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 3

The figure below illustrates that for neighboring leaves *i* and *j* sharing a parent node *m*, the following equality holds for every other leaf *k* in the tree:

$$\begin{aligned} d_{k,m} &= \frac{(d_{i,m} + d_{k,m}) + (d_{j,m} + d_{k,m}) - (d_{i,m} + d_{j,m})}{2} \\ &= \frac{d_{i,k} + d_{j,k} - d_{i,j}}{2} \end{aligned}$$

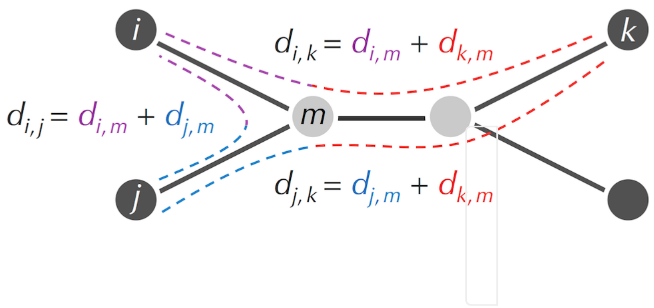


Figure: For neighboring leaves *i* and *j* and their parent node *m*, $d_{k,m} = \frac{(d_{i,k}+d_{j,k}-d_{i,j})}{2}$ for every other leaf *k* in the tree.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 4

Since *i*, *j*, and *k* are leaves, we can therefore compute the distance $d_{k,m}$ between nodes *k* and *m* in terms of elements of the additive distance matrix *D*:

$$d_{k,m} = \frac{(D_{i,k} + D_{j,k} - D_{i,j})}{2}$$

In the case when the parent *m* has degree 3 (as in the figure on the previous step), removing leaves *i* and *j* from the tree turns *m* into a leaf and thus reduces the total number of leaves (see figure below). This operation is equivalent to removing rows *i* and *j* as well as columns *i* and *j* from *D*, then adding a new row and column corresponding to their parent *m*, where the distances from *m* to other leaves are computed according to the above equation.

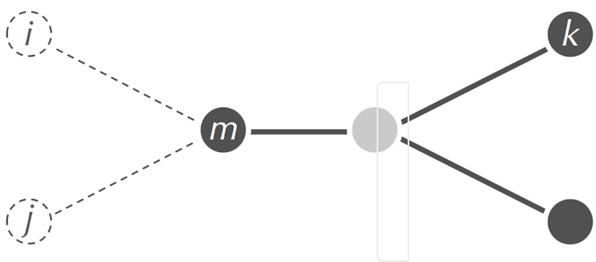


Figure: Removing leaves *i* and *j* from the tree turns *m* into a leaf (we assume that m has degree 3). The distances from this new leaf to any other leaf *k* can be recomputed as $d_{k,m} = \frac{(D_{i,k}+D_{j,k}-D_{i,j})}{2}$.

Exercise Break: We have just described how to reduce the size of the tree as well as the dimension of the distance matrix *D* if the parent node (*m*) has degree 3. Design a similar approach in the case that the degree of *m* is larger than 3.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 5

This discussion implies a recursive algorithm for the Distance-Based Phylogeny Problem:

- find a pair of neighboring leaves *i* and *j* by selecting the minimum element $D_{i,j}$ in the distance matrix;
- replace *i* and *j* with their parent, and recompute the distances from this parent to all other leaves as described above;
- solve the Distance-Based Phylogeny problem for the smaller tree;
- add the previously removed leaves *i* and *j* back to the tree.

Exercise Break: Apply this recursive approach to the additive distance matrix shown in the figure below. (Solve this exercise by hand.)

	i	***j***	***k***	***l***
i	0	13	21	22
j	13	0	12	13
k	21	12	0	13
l	22	13	13	0

Computing limb lengths

If you attempted the preceding exercise, then you were likely driven crazy. The reason why is that in the first step of our proposed algorithm, we assumed that a minimum element of an additive distance matrix corresponds to neighboring leaves. Yet as illustrated in figure below, this is not necessarily true! Thus, not only do we need a new approach to the Distance-Based Phylogeny Problem, but finding the animal coronavirus that is the smallest distance from SARS-CoV may not be the best way to identify the animal reservoir of SARS.

	i	***j***	***k***	***l***
i	0	13	21	22
j	13	0	12	13
k	21	12	0	13
l	22	13	13	0

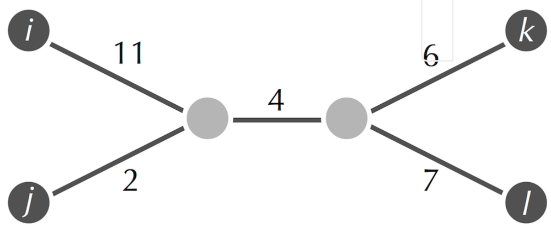


Figure: The distance matrix from the previous step along with the simple tree fitting this distance matrix. The two closest leaves in this tree (*j* and *k*) are not neighbors.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 7

Our proposed recursive approach may have failed, but using recursion was a good idea, and so we will explore a different recursive algorithm. Rather than looking for a *pair* of neighbors in *Tree*(*D*), we will instead reduce the size of the tree by trimming its leaves *one at a time*. Of course, we don't know *Tree*(*D*), and so we must somehow trim leaves in *Tree*(*D*) by analyzing the distance matrix.

Also, we previously attempted to compute all of the edge weights of *Tree(D)* from *D*. We will now address the more modest goal of computing only the lengths of limbs in *Tree(D)*. So, given a leaf *j* in a tree, we denote the length of the limb connecting *j* with its parent as *LimbLength*(*j*). Edges that are not limbs must connect two internal nodes and are therefore called **internal edges**.

Limb Length Problem: *Compute the length of a limb in a tree defined by an additive distance matrix.*

Input: An additive distance matrix *D* and an integer *j*.

Output: *LimbLength*(*j*), the length of the limb connecting leaf *j* to its parent.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 8

To compute *LimbLength*(*j*) for a given leaf *j*, note that because *Tree*(*D*) is simple, we know that *Parent*(*j*) has degree at least 3 (unless *Tree*(*D*) has only two nodes). We can therefore think of *Parent*(*j*) as dividing the other nodes of *Tree*(*D*) into at least three **subtrees**, or smaller trees that would remain if we were to remove *Parent*(*j*) along with any edges connecting it to other nodes (see figure below). Because *j* is a leaf, it must belong to a subtree by itself; we call this subtree T_j. This brings us to the following result.

Limb Length Theorem: Given an additive matrix *D* and a leaf *j*, *LimbLength*(*j*) is equal to the minimum value of $(D_{i,j} + D_{j,k} - D_{i,k})/2$ over all leaves *i* and *k*.

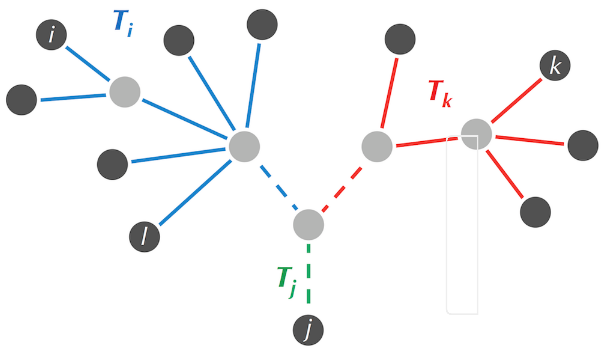
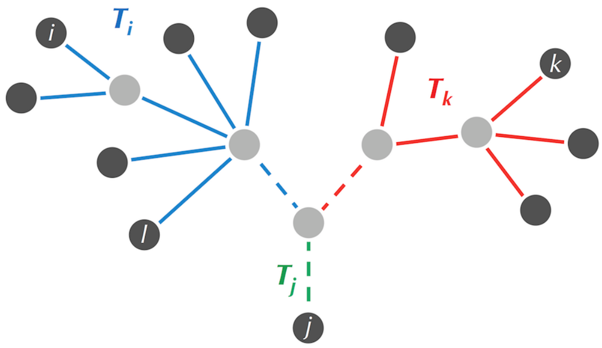


Figure: A simple tree with selected leaves *i*, *j*, *k*, and *l*. Removing the parent of *j* (along with the three dashed edges connecting it to other nodes) would separate this tree into three subtrees, whose edges are shown in different colors. Leaves *i* and *l* belong to $\boldsymbol{T_i}$, whereas leaf *k* belongs to $\boldsymbol{T_k}$. Leaf *j* belongs to $\boldsymbol{T_j}$, which contains a single node.

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 9



Proof of Limb Length Theorem: A given pair of leaves can belong to the same subtree or to different subtrees in the figure on the previous step, reproduced above. So first assume that leaves *i* and *k* belong to different subtrees T_i and T_k. Because *Parent*(*j*) is on the path connecting *i* to *k*, it follows that

$$d_{i,j} = d_{i,Parent(j)} + LimbLength(j)$$

$$d_{j,k} = d_{k,Parent(j)} + LimbLength(j)$$

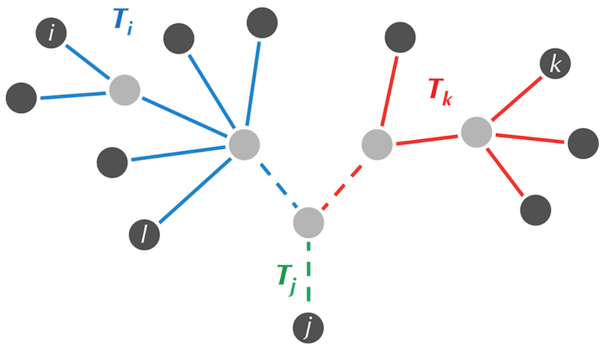
Adding these two equations yields

$$d_{i,j} + d_{j,k} = d_{i,Parent(j)} + d_{k,Parent(j)} + 2 \cdot LimbLength(j)$$

Because $d_{i,Parent(j)} + d_{k,Parent(j)}$ is equal to $d_{i,k}$, it follows that

$$LimbLength(j) = \frac{d_{i,j} + d_{j,k} - d_{i,k}}{2} = \frac{D_{i,j} + D_{j,k} - D_{i,k}}{2}.$$

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 10



On the other hand, assume that leaves *i* and *l* belong to the same subtree (see the figure on the previous step). Then the path from *i* to *l* does not pass through *Parent*(*j*), and so we have the *inequality*

$$d_{i,Parent(j)} + d_{l,Parent(j)} \geq d_{i,l}\,.$$

Combining this with the equation

$$d_{i,j} + d_{j,l} = d_{i,Parent(j)} + d_{l,Parent(j)} + 2 \cdot LimbLength(j)$$

yields that

$$LimbLength(j) = \frac{d_{i,j} + d_{j,l} - (d_{i,Parent(j)} + d_{l,Parent(j)})}{2} \leq \frac{d_{i,j} + d_{j,l} - d_{i,l}}{2} = \frac{D_{i,j} + D_{j,l} - D_{i,l}}{2}.$$

As a result of this discussion, *LimbLength*(*j*) must be less than or equal to $(D_{i,j} + D_{j,k} - D_{i,k})/2$ for any choice of leaves *i* and *k*. Because we can always find leaves *i* and *k* belonging to different subtrees (why?), it follows that *LimbLength*(*j*) is equal to the minimum value of $(D_{i,j} + D_{j,k} - D_{i,k})/2$ over all choices of *i* and *k*. □

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 11

We now have an algorithm for solving the Limb Length Problem. For each *j*, we can compute *LimbLength*(*j*) by finding the minimum value of $(D_{i,j} + D_{j,k} - D_{i,k})/2$ over all pairs of leaves *i* and *k*.

CODE CHALLENGE: Solve the Limb Length Problem.

Input: An integer *n*, followed by an integer *j* between 0 and *n*, followed by a space-separated additive distance matrix *D* (whose elements are integers).

Output: The limb length of the leaf in *Tree*(*D*) corresponding to the *j*-th row of this distance matrix (use 0-based indexing).

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Limb_Length.txt)

Sample Input:

```
4
1
0	13	21	22
13	0	12	13
21	12	0	13
22	13	13	0
```

Sample Output:

```
2
```

Time Limit: `5 mins`

To pass this quiz please visit stepic.org/lesson/-10329/step/11

Toward An Algorithm for Distance-Based Phylogeny Construction | Step 12

Exercise Break: The algorithm proposed on the previous step computes *LimbLength*(j) in $O(n^2)$ time (where n is the number of leaves). Design an algorithm that computes *LimbLength*(j) in $O(n)$ time.

Additive Phylogeny | Step 1

Trimming the tree

Since we now know how to find the length of any limb in *Tree*(*D*), we can construct *Tree*(*D*) recursively using the algorithm illustrated in the figure below and described on the next few steps.

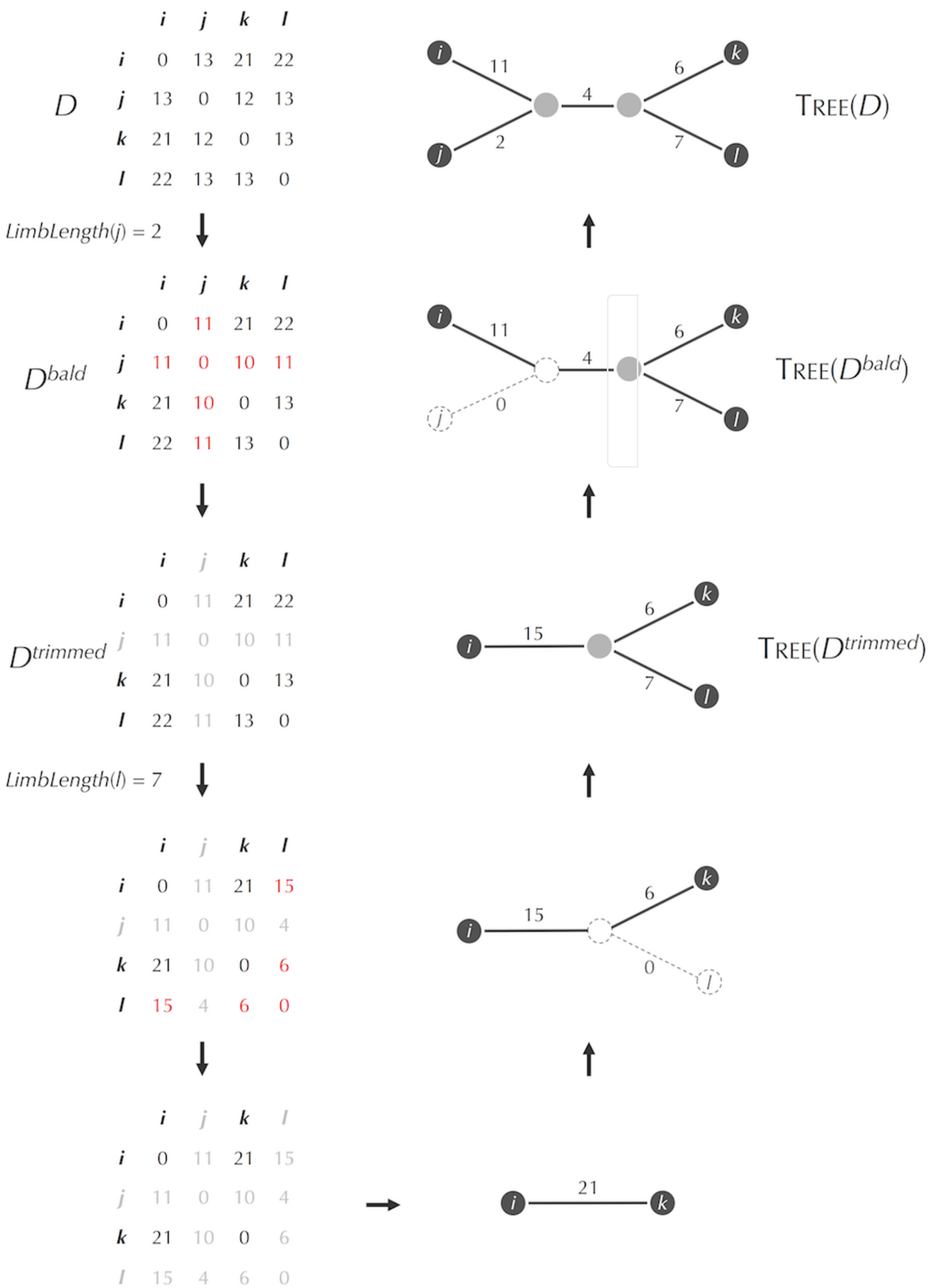
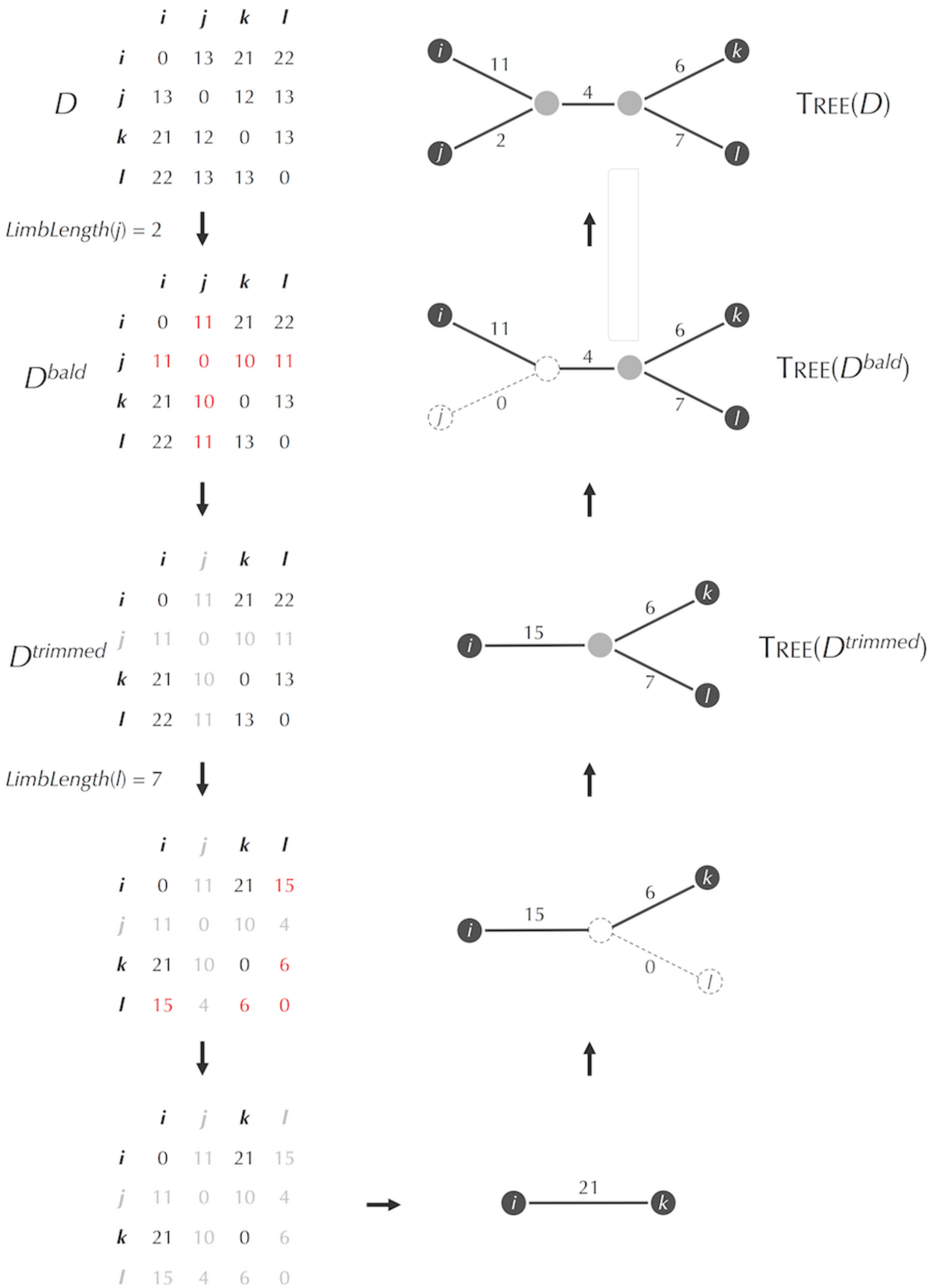


Figure: Converting the additive distance matrix from Figure 1.9 (left) into the simple tree fitting this matrix from Figure 1.10. On the left side, we first compute *LimbLength*(*j*) = 2, and then subtract 2 from the elements in the *i*-th row and *j*-th column of *D* to obtain D^{bald} (updated values are shown in red). Removing this row and column yields a 3 x 3 distance matrix $D^{trimmed}$. We find that *LimbLength*(*l*) = 7 in $D^{trimmed}$ and subtract 7 from the elements in its *l*-th row and *l*-th column. Graying out this row and column yields a 2 x 2 distance matrix. On the right side, we can fit this 2 x 2 distance matrix to a tree consisting of a single edge. By finding the attachment points of removed limbs (shown on the left), we reconstruct *Tree*($D^{trimmed}$) and then *Tree*(*D*).

Additive Phylogeny | Step 2

First, imagine that we already know *Tree*(*D*), and pick an arbitrary leaf *j*. We will trim the limb of *j* by reducing its length by *LimbLength*(*j*). Because we do not know *Tree*(*D*) in advance, we need to represent trimming the leaf *j* in terms of the distance matrix *D*. To do so, we first subtract *LimbLength*(*j*) from each off-diagonal element in the *j*-th row and *j*-th column of *D* to obtain a matrix D^{bald} for which the limb of *j* has become a **bald limb**, or a limb of length 0 (see the figure on the previous step). We will further assume that a bald limb has disappeared from the tree entirely. In terms of the distance matrix, ignoring a bald limb means removing the *j*-th row and column from *D* to produce a smaller (*n* - 1) x (*n* - 1) distance matrix $D^{trimmed}$.

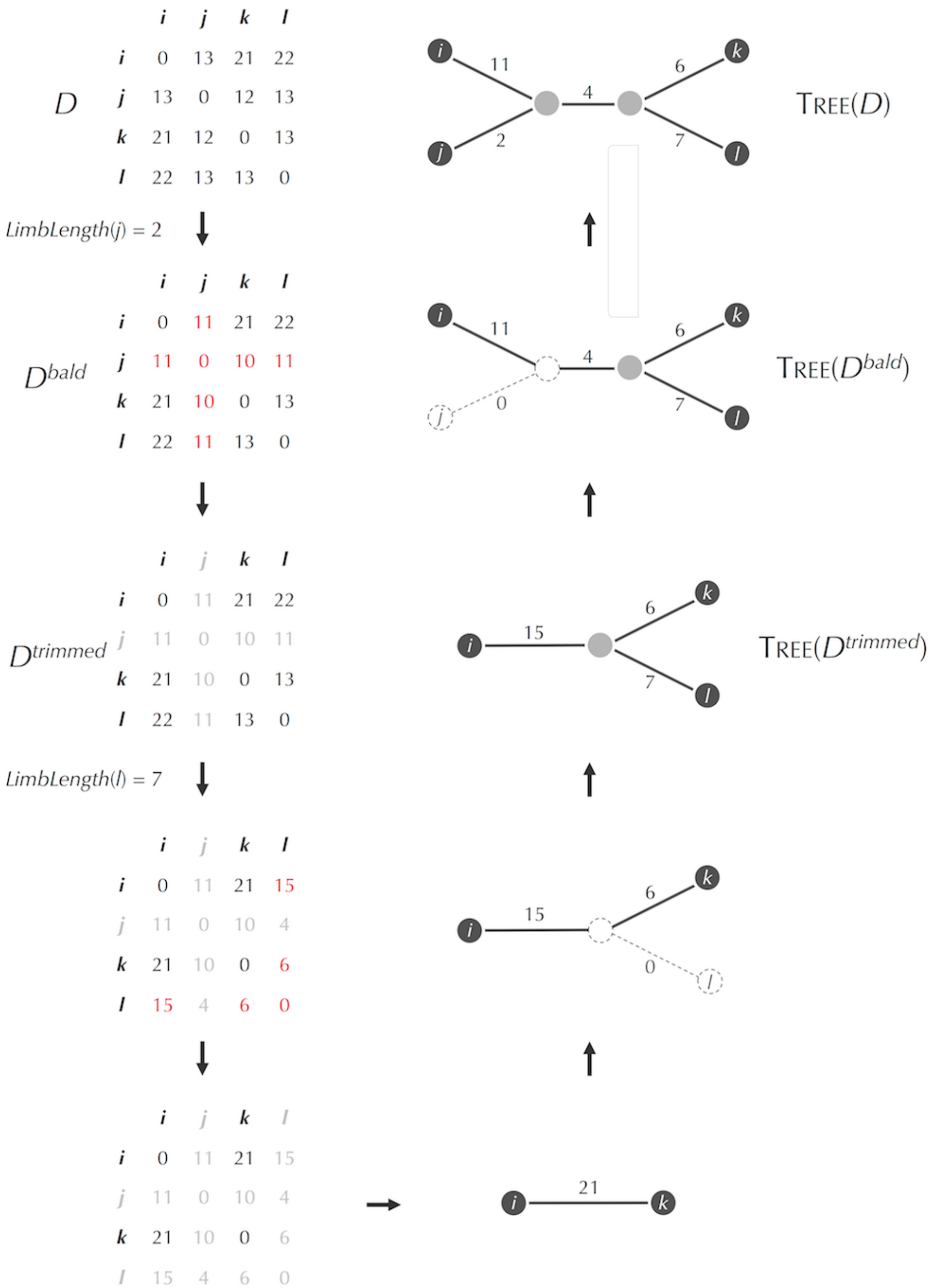


Additive Phylogeny | Step 3

We can now recursively find *Tree*(D) in four steps:

- pick an arbitrary leaf j, compute *LimbLength*(j), and construct the distance matrix $D^{trimmed}$;
- solve the Distance-Based Phylogeny Problem for $D^{trimmed}$;
- identify the point in *Tree*($D^{trimmed}$) where leaf j should be attached in *Tree*(D);
- add a limb of length *LimbLength*(j) growing from this attachment point in *Tree*($D^{trimmed}$) to form *Tree*(D).

STOP and Think: When adding leaf j back to *Tree*($D^{trimmed}$), how would you find its attachment point?



Attaching a limb

To find the attachment point of a leaf *j* in *Tree*($D^{trimmed}$), consider *Tree*($D^{trimmed}$), which is the same as *Tree*(*D*) except that *LimbLength*(*j*) = 0. From the Limb Length Theorem, we know that there must be leaves *i* and *k* in *Tree*($D^{trimmed}$) such that

$$\frac{D^{bald}_{i,j} + D^{bald}_{j,k} - D^{bald}_{i,k}}{2} = 0,$$

which implies that

$$D^{bald}_{i,k} = D^{bald}_{i,j} + D^{bald}_{j,k}.$$

Thus, the attachment point for leaf *j* must be located at distance $D^{bald}_{i,j}$ from leaf *i* on the path connecting *i* and *k* in the trimmed tree. This attachment point may occur at an existing node, in which case we connect *j* to this node. On the other hand, the attachment point for *j* may occur along an edge, in which case we place a new node at the attachment point and connect *j* to it.

Additive Phylogeny | Step 5

An algorithm for distance-based phylogeny reconstruction

The preceding discussion results in the following recursive algorithm, called **AdditivePhylogeny**, for finding the simple tree fitting an *n* x *n* additive distance matrix *D*. We assume that you have already implemented a program **Limb**(*D, j*) that computes *LimbLength*(*j*) for a leaf *j* based on the distance matrix *D*. Rather than selecting an arbitrary leaf *j* for trimming from *Tree*(*D*), **AdditivePhylogeny** selects leaf *n* (corresponding to the last row and column of *D*).

AdditivePhylogeny(*D*, *n*)
    if *n* = 2
        return the tree consisting of a single edge of length $D_{1,2}$
    limbLength ← **Limb**(*D, n)*
    for *j* ← 1 to *n* - 1
        $D_{j,n} \leftarrow D_{j,n}$ - *limbLength*
        $D_{n,j} \leftarrow D_{j,n}$
    (*i,n,k*) ← three leaves such that $D_{i,k} = D_{i,n} + D_{n,k}$
    $x \leftarrow D_{i,n}$
    remove *n*-th row and column from *D*
    T ← **AdditivePhylogeny**(*D*, *n* - 1)
    v ← the (potentially new) node in *T* located at distance *x* from leaf *i* on the path between *i* and *k*
    add leaf *n* back to *T* by creating a limb (*v*, *n*) of length *limbLength*
    return *T*

Additive Phylogeny | Step 6

CODE CHALLENGE: Implement **AdditivePhylogeny** to solve the Distance-Based Phylogeny Problem.

Input: An integer *n* followed by a space-separated *n* x *n* distance matrix.

Output: A weighted adjacency list for the simple tree fitting this matrix.

Note on formatting: The adjacency list must have consecutive integer node labels starting from 0. The *n* leaves must be labeled 0, 1, ..., *n* - 1 in order of their appearance in the distance matrix. Labels for internal nodes may be labeled in any order but must start from *n* and increase consecutively.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/evolution/Additive_Phylogeny.txt)

Sample Input:

```
4
0	13	21	22
13	0	12	13
21	12	0	13
22	13	13	0
```

Sample Output:

```
0->4:11
1->4:2
2->5:6
3->5:7
4->0:11
4->1:2
4->5:4
5->4:4
5->3:7
5->2:6
```

Time Limit: `5 mins`

To pass this quiz please visit stepic.org/lesson/-10330/step/6

Additive Phylogeny | Step 7

STOP and Think: Consider these questions about **AdditivePhylogeny**.

- Estimate its running time.
- Although it may seem that **AdditivePhylogeny** would construct a tree for any matrix, this is not the case. What goes wrong if you apply **AdditivePhylogeny** to the non-additive distance matrix in the figure below?
- Modify **AdditivePhylogeny** to develop an algorithm that checks whether a given distance matrix is additive. Then, apply this test to the distance matrix for coronavirus Spike proteins shown on the next step. Is this matrix additive?

Although the previous question suggests that **AdditivePhylogeny** can be modified to determine whether a given distance matrix is additive, there exists an even simpler way to check for additivity (see DETOUR: The Four Point Condition (http://stepic.org/lesson/Detour-The-Four-Point-Condition-10340/)).

	i	***j***	***k***	***l***
i	0	3	4	3
j	3	0	4	5
k	4	4	0	2
l	3	5	2	0

Constructing an evolutionary tree of coronaviruses

By the end of 2003, bioinformaticians had sequenced many coronaviruses from a variety of birds and mammals, from which we obtain the distance matrix below based on a multiple alignment of Spike proteins.

	Cow	Pig	Horse	Mouse	Dog	Cat	Turkey	Civet	Human
Cow	0	295	300	524	1077	1080	978	941	940
Pig	295	0	314	487	1071	1088	1010	963	966
Horse	300	314	0	472	1085	1088	1025	965	956
Mouse	524	487	472	0	1101	1099	1021	962	965
Dog	1076	1070	1085	1101	0	818	1053	1057	1054
Cat	1082	1088	1088	1098	818	0	1070	1085	1080
Turkey	976	1011	1025	1021	1053	1070	0	963	961
Civet	941	963	965	962	1057	1085	963	0	16
Human	940	966	956	965	1054	1080	961	16	0

Figure: The distance matrix based on pairwise alignment of Spike proteins from coronaviruses extracted from various animals. The distance between each pair of sequences was computed as the total number of mismatches and indels in their optimal alignment.

Additive Phylogeny | Step 9

Although you now understand the perils of concluding that the minimum element of the distance matrix corresponds to a pair of neighbors, common sense tells us with a glance at the coronavirus distance matrix that the civet must be the animal reservoir of SARS. This information led researchers to hypothesize that inadequate preparation of meat from palm civets (see the photo below) in the Guangdong region of China may have caused the SARS outbreak.

	Cow	Pig	Horse	Mouse	Dog	Cat	Turkey	Civet	Human
Cow	0	295	300	524	1077	1080	978	941	940
Pig	295	0	314	487	1071	1088	1010	963	966
Horse	300	314	0	472	1085	1088	1025	965	956
Mouse	524	487	472	0	1101	1099	1021	962	965
Dog	1076	1070	1085	1101	0	818	1053	1057	1054
Cat	1082	1088	1088	1098	818	0	1070	1085	1080
Turkey	976	1011	1025	1021	1053	1070	0	963	961
Civet	941	963	965	962	1057	1085	963	0	16
Human	940	966	956	965	1054	1080	961	16	0

Figure: The palm civet. Courtesy Praveenp, Wikimedia Commons user.

Additive Phylogeny | Step 10

However, before rushing to this conclusion, you may like to read DETOUR: Did Bats Give Us SARS (http://stepic.org/lesson/Detour-Did-Bats-Give-Us-SARS-10341/)? to see why the exact history of interspecies viral transfer is often difficult to trace. In fact, some studies have suggested that humans first received SARS from bats, which later gave the virus to palm civets, which then transmitted the disease back to humans. The civet was identified as the animal reservoir of SARS in 2003 in part because SARS viruses from other potential suspects, including bats, had not yet been sequenced.

STOP and Think: It turns out that most distance matrices constructed from real data (including the coronavirus distance matrix) are non-additive. Why do you think that this is the case?

Additive Phylogeny | Step 11

Since the real distance matrix for SARS-like coronaviruses is non-additive, we will cheat a bit and slightly modify it to make it additive so that you can apply **AdditivePhylogeny** to it (see the figure below).

	Cow	Pig	Horse	Mouse	Dog	Cat	Turkey	Civet	Human
Cow	0	295	306	497	1081	1091	1003	956	954
Pig	295	0	309	500	1084	1094	1006	959	957
Horse	306	309	0	489	1073	1083	995	948	946
Mouse	497	500	489	0	1092	1102	1014	967	965
Dog	1081	1084	1073	1092	0	818	1056	1053	1051
Cat	1091	1094	1083	1102	818	0	1066	1063	1061
Turkey	1003	1006	995	1014	1056	1066	0	975	973
Civet	956	959	948	967	1053	1063	975	0	16
Human	954	957	946	965	1051	1061	973	16	0

Figure: An additive modification of the Spike protein distance matrix.

Exercise Break: Construct the simple tree fitting this distance matrix.

Download Matrix

(http://bioinformaticsalgorithms.com/data/realdatasets/Evolution/coronavirus_distance_matrix_additive.txt)

Detour: When Did HIV Jump from Primates to Humans? | Step 1

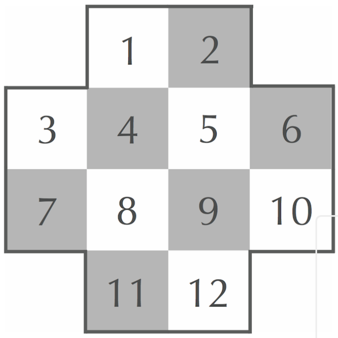
Scientists became aware that HIV caused AIDS in the 1980s, at a time when the virus was still rare. They immediately started looking for earlier cases of HIV infection in various medical records and found HIV in a blood sample taken from a Congolese patient in 1959. Genetic studies then revealed that HIV is closely related to Simian Immunodeficiency Virus (SIV), which infects primates, but it remained unclear how and when SIV entered the human population and evolved into HIV. The most popular current hypothesis is that SIV evolved into HIV when hunters killed monkeys to sell their meat and were exposed to the animals' blood. The virus circulating in the blood had further entered cuts in the hunters' skin, mutated, and later adapted to humans.

By finding regions in the viral genome that mutate at a roughly constant rate over time, researchers can infer a timeline of HIV evolution. Using this "molecular clock", biologists estimated the time points when the various subtypes of SIV jumped from primates to human: for HIV groups A, B, M, and O, these transitions have been estimated at 1940, 1945, 1908, and 1920, respectively (the timing of the jump for group N is currently unknown). By sequencing fecal samples of wild primates, biologists have even been able to locate populations of chimpanzees and sooty mangabeys whose SIVs are direct ancestors of HIV groups.

Detour: An Introduction to Graphs | Step 1

The use of the word “graph” in this book is different from its use in high school mathematics; we do not mean a chart of data. You can think of a graph as a diagram showing cities connected by roads.

The figure below shows a 4 × 4 chessboard with the four corner squares removed. A knight can move two steps in any of four directions (left, right, up, and down) followed by one step in a perpendicular direction. For example, a knight at square 1 can move to square 7 (two down and one left), square 9 (two down and one right), or square 6 (two right and one down).



STOP and Think: Can a knight travel around this board, pass through each square exactly once, and return to the same square it started on?

Detour: An Introduction to Graphs | Step 2

We further represent each of the 12 squares at the chessboard as a node. Two nodes are connected by an edge if a knight can move between them in a single step. For example, node 1 is connected to nodes 6, 7, and 9. Connecting nodes in this manner produces a "Knight Graph" consisting of 12 nodes and 16 edges.

We can describe a graph by its set of nodes and edges, where every edge is written as the pair of nodes that it connects. The graph in the second panel of the figure below is described by the node set:

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
```

and the edge set:

```
{1,6}  {1,7}  {1,9} {2,3}  {2,8} {2,10} {3,9}  {3,11}
{4,10} {4,12} {5,7} {5,11} {6,8} {6,12} {7,12} {10,11}
```

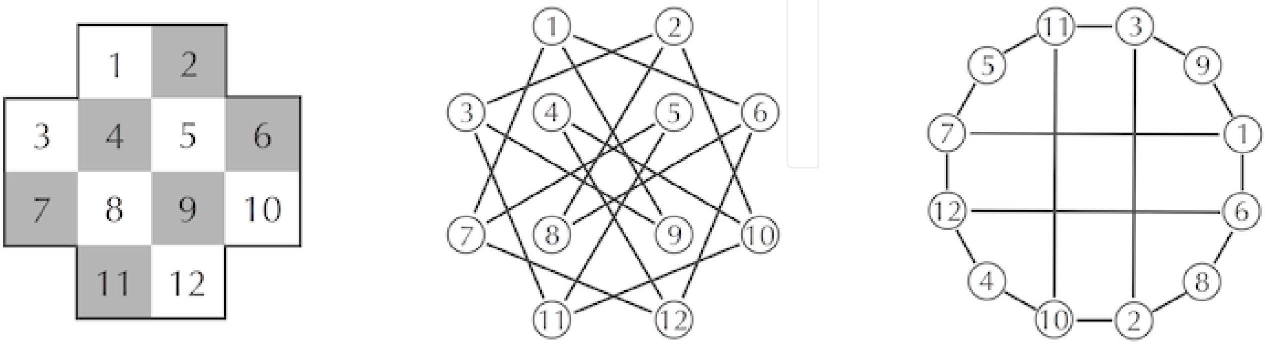


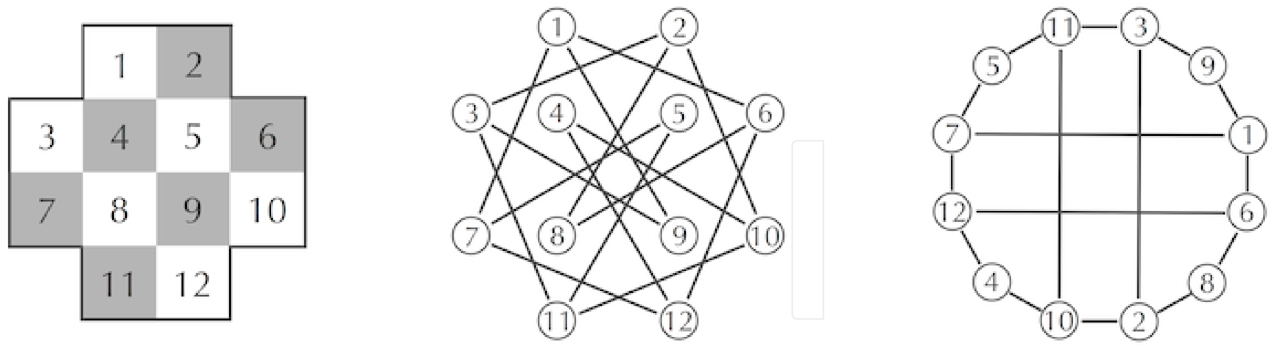
Figure: (Left) A hypothetical chessboard with squares numbered. (Middle) The Knight Graph represents each square by a node and connects two nodes with an edge if a knight can travel between the respective squares in only one move. (Right) An equivalent representation of the Knight Graph.

The way the graph is drawn is irrelevant; two graphs with the same node and edge sets are equivalent, even if the particular pictures that represent the graph are different. The only thing that is important is which nodes are connected and which are not. Therefore, the graph in the third panel of the figure above is identical to the graph in the second panel.

Detour: An Introduction to Graphs | Step 3

A **path** in a graph is a sequence of edges, where each successive edge begins at the node where the previous edge ends. For example, the path 8 → 6 → 1 → 9 in the Knight Graph starts at node 8, ends at node 9, and consists of 3 edges. Paths that start and end at the same node are referred to as **cycles**. The cycle 3 → 2 → 10 → 11 → 3 starts and ends at node 3 and consists of 4 edges.

The nodes and edges of the graph shown in the third panel of the figure from the previous step (reproduced below) reveal a cycle that visits every node in the Knight Graph once and describes a sequence of knight moves that visit every square exactly once.



EXERCISE BREAK: How many knight's tours exist for the above chessboard? (A knight's tour must start and end at square 1.)

To pass this quiz please visit stepic.org/lesson/-208/step/3

Detour: An Introduction to Graphs | Step 4

The number of edges incident to a given node v is called the **degree** of *v*. For example, node 1 in the Knight Graph has degree 3, while node 5 has degree 2. The sum of degrees of all 12 nodes is, in this case, 32 (eight nodes have degree 3 and four nodes have degree 2), which is twice the number of edges in the graph.

STOP and Think: Can you connect seven phones in such a way that each is connected to exactly three others?

Many bioinformatics problems analyze **directed graphs**, in which every edge is directed from one node to another, as shown by the arrows in Figure 54. You can think of a directed graph as a diagram showing cities connected by one-way roads. Every node in a directed graph is characterized by indegree (the number of incoming edges) and outdegree (the number of outgoing edges).

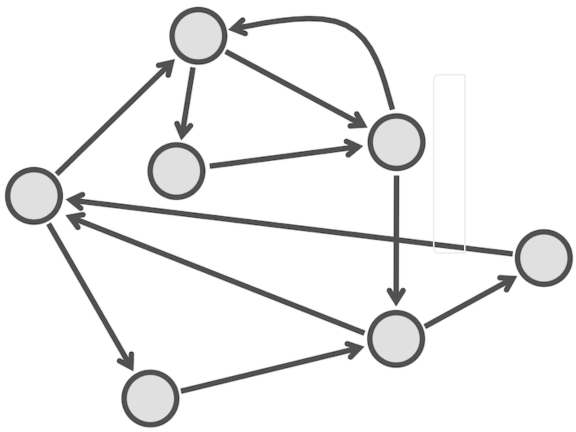


Figure: A directed graph.

Detour: An Introduction to Graphs | Step 5

STOP and Think: Prove that for every directed graph, the sum of indegrees of all nodes is equal to the sum of outdegrees of all nodes.

An undirected graph is **connected** if every two nodes have a path connecting them. Disconnected graphs can be partitioned into disjoint connected components.

Detour: Searching for a Tree Fitting a Distance Matrix | Step 1

Every 3 × 3 matrix D is additive. To see why, note that there is only one tree with three leaves. We will denote the leaves of this tree as 1, 2, and 3, and its internal node as c. As illustrated in the figure below, the lengths of edges in this tree must satisfy the following three equations:

$$d_{1,c} + d_{2,c} = D_{1,2} \quad d_{1,c} + d_{3,c} = D_{1,3} \quad d_{2,c} + d_{3,c} = D_{2,3}$$

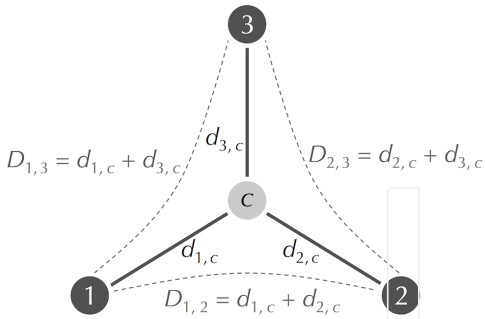


Figure: The tree with three leaves (labeled 1, 2, and 3) and an internal node (labeled c). The distances between leaves ($D_{1,2}$, $D_{1,3}$, and $D_{2,3}$) uniquely define the lengths of edges ($d_{1,c}$, $d_{2,c}$, and $d_{3,c}$).

Detour: Searching for a Tree Fitting a Distance Matrix | Step 2

Solving this system of equations for $d_{1,c}$, $d_{2,c}$, and $d_{3,c}$ yields the following formulas for the lengths of edges in terms of values of the matrix D:

$$d_{1,c} = (D_{1,2} + D_{1,3} - D_{2,3}) / 2$$

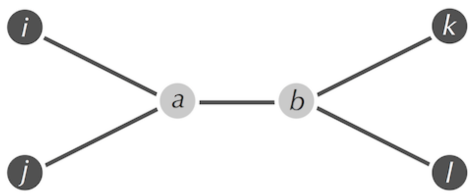
$$d_{2,c} = (D_{2,1} + D_{2,3} - D_{1,3}) / 2$$

$$d_{3,c} = (D_{3,1} + D_{3,2} - D_{1,2}) / 2$$

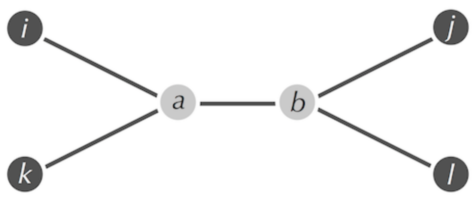
Detour: Searching for a Tree Fitting a Distance Matrix | Step 3

The figure on the next step illustrates an attempt to fit a distance matrix to all possible unrooted trees with four leaves. Each such tree leads to a system of six linear equations (in either four or five variables) that does not have a solution. Thus, this distance matrix must be non-additive.

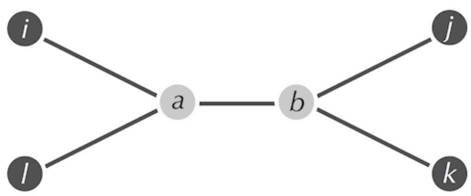
	i	***j***	***k***	***l***
i	0	3	4	3
j	3	0	4	5
k	4	4	0	2
l	3	5	2	0



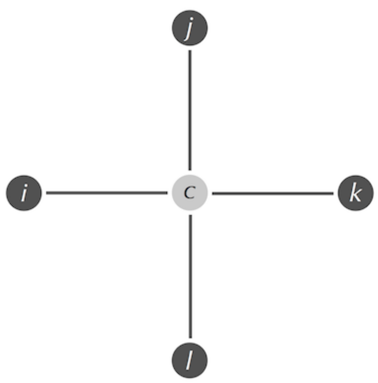
$$\begin{aligned} d_{i,a} + d_{j,a} &= D_{i,j} = 3 \\ d_{i,a} + d_{a,b} + d_{k,b} &= D_{i,k} = 4 \\ d_{i,a} + d_{a,b} + d_{l,b} &= D_{i,l} = 3 \\ d_{j,a} + d_{a,b} + d_{k,b} &= D_{j,k} = 4 \\ d_{j,a} + d_{a,b} + d_{l,b} &= D_{j,l} = 5 \\ d_{k,b} + d_{l,b} &= D_{k,l} = 2 \end{aligned}$$



$$\begin{aligned} d_{i,a} + d_{a,b} + d_{j,b} &= D_{i,j} = 3 \\ d_{i,a} + d_{k,a} &= D_{i,k} = 4 \\ d_{i,a} + d_{a,b} + d_{l,b} &= D_{i,l} = 3 \\ d_{j,b} + d_{a,b} + d_{k,a} &= D_{j,k} = 4 \\ d_{j,b} + d_{l,b} &= D_{j,l} = 5 \\ d_{k,a} + d_{a,b} + d_{l,b} &= D_{k,l} = 2 \end{aligned}$$



$$\begin{aligned} d_{i,a} + d_{a,b} + d_{j,b} &= D_{i,j} = 3 \\ d_{i,a} + d_{a,b} + d_{k,b} &= D_{i,k} = 4 \\ d_{i,a} + d_{l,a} &= D_{i,l} = 3 \\ d_{j,b} + d_{k,b} &= D_{j,k} = 4 \\ d_{j,b} + d_{a,b} + d_{l,b} &= D_{j,l} = 5 \\ d_{k,a} + d_{a,b} + d_{l,a} &= D_{k,l} = 2 \end{aligned}$$



$$\begin{aligned} d_{i,c} + d_{j,c} &= D_{i,j} = 3 \\ d_{i,c} + d_{k,c} &= D_{i,k} = 4 \\ d_{i,c} + d_{l,c} &= D_{i,l} = 3 \\ d_{j,c} + d_{k,c} &= D_{j,k} = 4 \\ d_{j,c} + d_{l,c} &= D_{j,l} = 5 \\ d_{k,c} + d_{l,c} &= D_{k,l} = 2 \end{aligned}$$

Figure (Top) A 4 × 4 distance matrix *D*. (Left) All four trees containing four leaves (labeled 1, 2, 3, 4). (Right) Each attempt to fit *D* to a tree results in a system of six linear equations. Because none of these systems has a solution, *D* must be non-additive.

Detour: Searching for a Tree Fitting a Distance Matrix | Step 4

The reason we failed to fit a tree to the 4 × 4 distance matrix on the previous step is partly because the number of equations was larger than the number of variables ($d_{i,j}$) for each tree (a fact that will also hold for larger values of n). If the number of equations in a linear system of equations is smaller than or equal to the number of variables in the system, then a solution usually exists, whereas if the number of equations exceeds the number of variables, there is usually no solution. However, there are exceptions in both directions. Consult an introductory linear algebra text for more details.

Detour: The Four Point Condition | Step 1

The **four point condition** gives an alternative way to determine if a matrix is additive. Consider the tree containing only four leaves in the figure below. For this tree, observe that

$$d_{i,j} + d_{k,l} \leq d_{i,k} + d_{j,l} = d_{i,l} + d_{j,k}$$

because the first sum is the sum of lengths of all edges in the tree *minus* the length of the internal edge, while the last two sums are equal to the sum of lengths of all edges in the tree *plus* the length of the internal edge.

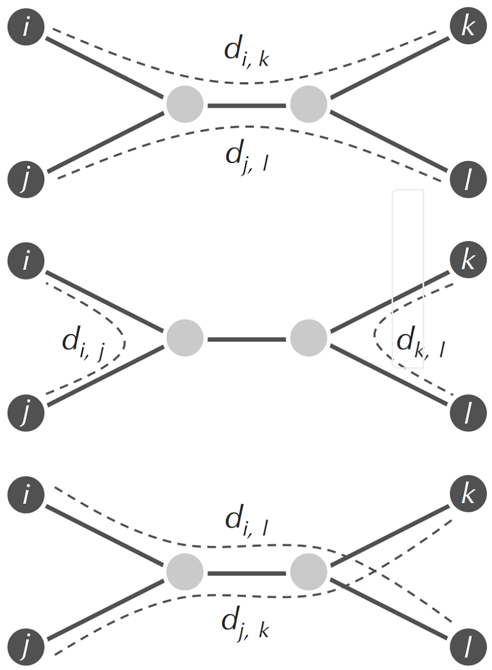


Figure: Three pairs of paths through a tree with four leaves. The paths in the top left and top right traverse the same edges, and so $d_{i,k} + d_{j,l} = d_{i,l} + d_{j,k}$. Furthermore, $d_{i,j} + d_{k,l}$ must be less than or equal to these two sums, since it does not traverse the internal edge of the tree, as shown in the tree on the bottom.

Detour: The Four Point Condition | Step 2

Indeed, for any quartet of leaves (i, j, k, l) in a tree of arbitrary size, if we compute the three sums

$$d_{i,j} + d_{k,l} \qquad d_{i,k} + d_{j,l} \qquad d_{i,l} + d_{j,k}$$

then we will find that two of the sums are equal and that the third sum is less than or equal to the other two sums. In terms of an $n \times n$ distance matrix, we say that a quartet of indices (i, j, k, l) satisfies the **four point condition** if two of the following sums are equal, and the third sum is less than or equal to the other two sums:

$$D_{i,j} + D_{k,l} \qquad D_{i,k} + D_{j,l} \qquad D_{i,l} + D_{j,k}$$

Four Point Theorem: A distance matrix is additive if and only if the four point condition holds for every quartet (i, j, k, l) of indices of this matrix.

Exercise Break: Prove the Four Point Theorem.

Detour: The Four Point Condition | Step 3

The Four Point Theorem provides us with an alternative way of determining whether a given distance matrix is additive, since we can simply check whether the four point condition holds for each quartet of indices of the matrix. If there is a single such quartet violating the four point condition, then the distance matrix must be non-additive.

Exercise Break: Compare the running time of this proposed method with **AdditivePhylogeny** for determining distance matrix additivity.

STOP and Think: Verify that the four point condition does not hold for the single quartet of indices from the non-additive distance matrix in the figure below.

	i	***j***	***k***	***l***
i	0	3	4	3
j	3	0	4	5
k	4	4	0	2
l	3	5	2	0

Detour: Did Bats Give Us SARS? | Step 1

During the search for the animal reservoir of the SARS virus, biologists discovered infected palm civets at a live animal market in China. Meat from these animals is often added to “dragon-tiger-phoenix soup”, an expensive Cantonese dish. The discovery did not greatly change the infected civets’ fate: instead of ending up in soup, they were made into SARS scapegoats and slaughtered. Yet when further searches failed to identify more SARS-infected civets, biologists started to wonder whether palm civets really were the original source of SARS. In 2005, they discovered a SARS-like virus in Chinese horseshoe bats (figure below). The bats turned out to be SARS-CoV carriers, but they can probably only pass the virus to humans through intermediate hosts. Since bat meat is considered a delicacy and is also used in traditional Chinese medicine, bats had plenty of chances to come in close contact with civets at overcrowded live animal markets.

Figure: The horseshoe bat. Courtesy Praveenp, Wikimedia Commons user.

Detour: Did Bats Give Us SARS? | Step 2

When biologists constructed the evolutionary tree of coronaviruses from bats, civets, and humans (figure below), they found that both the civet and human variants of SARS- CoV are nested within a bat virus phylogeny.

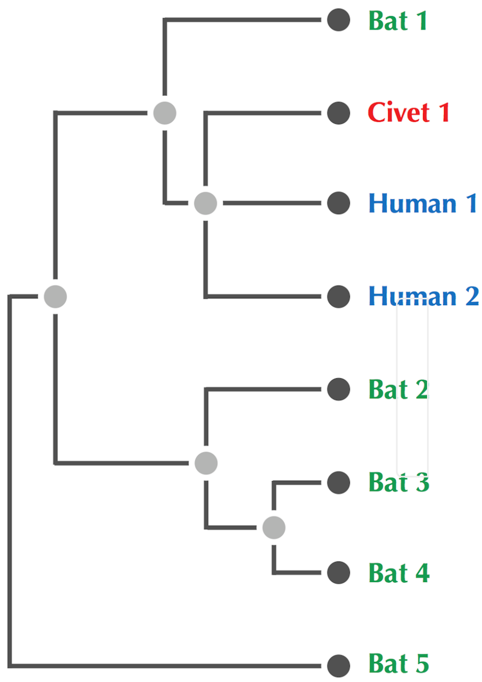


Figure: An evolutionary tree of coronaviruses from bats, civets, and humans.

Detour: Did Bats Give Us SARS? | Step 3

Even before sequencing SARS-CoV, biologists knew of two human coronaviruses (HCoV-229E and HCoV-OC43) but considered them harmless. After the SARS outbreak in 2003, the search for new human coronaviruses intensified, and in 2005, biologists identified two more coronaviruses, NL63 and HKU1. However, the next deadly coronavirus did not appear until 2012, when a SARS-like coronavirus emerged in Saudi Arabia, called **Middle East Respiratory Syndrome (MERS)**, and generated international headlines when it spread to other countries.

Although researchers initially believed that camels were to blame for MERS — many Saudis consume unpasteurized camel milk — most afflicted patients had not come in contact with camels. Yet when researchers tested a coronavirus fragment taken from bat found just a few miles from one of the first MERS patient’s homes, they found this fragment to be a perfect match with the patient’s virus sample.